MATH547 PROJECT: UNDERSTANDING NEURAL TANGENT KERNELS

Sampad B Mohanty
Department of Computer Science
University of Southern California
Los Angeles, CA 15213, USA
sbmohant@usc.edu

ABSTRACT

This report surveys the Neural Tangent Kernel (NTK) framework, which characterizes the behavior of infinitely wide neural networks during training. We review the necessary mathematical foundations, beginning with Reproducing Kernel Hilbert Spaces (RKHS) for scalar-valued and vector-valued functions. We then discuss Gaussian Processes and their connection to randomly initialized neural networks through the Neural Network Gaussian Process (NNGP) kernel. Building on these foundations, we present the NTK derivation, showing how gradient descent in overparameterized networks linearizes around initialization, reducing non-convex optimization to kernel gradient descent with a fixed kernel. This explains the lazy learning phenomenon, where training loss decreases to zero while parameters remain close to their initial values. We discuss connections between NTK theory, vector-valued RKHS, and Gaussian Process regression. Finally, we note both the insights provided by NTK—particularly regarding convergence and initialization—and its limitations. While NTK rigorously characterizes the infinite-width limit, finite-width networks often outperform kernel methods in practice, suggesting that feature learning at realistic widths operates beyond the lazy regime.

1 INTRODUCTION

Modern deep neural networks exhibit a remarkable empirical phenomenon: despite operating in highly non-convex parameter spaces with millions or billions of parameters, simple gradient-based optimization reliably converges to near-zero training error while maintaining strong generalization. Understanding this behavior has been a central challenge in machine learning theory. The Neural Tangent Kernel (NTK) framework Jacot et al. (2018) provides a powerful theoretical lens for this puzzle by revealing that, in the limit of infinite width, neural network training can be understood through the well-established theory of kernel methods.

The key insight is both surprising and elegant. As network width grows, gradient descent dynamics *linearize around initialization*, transforming the complex non-convex optimization problem into kernel gradient descent governed by a deterministic, essentially fixed kernel—the NTK. In this regime, training an infinitely wide neural network becomes mathematically equivalent to performing kernel regression in a Reproducing Kernel Hilbert Space (RKHS), where the kernel is determined entirely by the network architecture and initialization scheme. This *lazy training* regime—where parameters move minimally from their initial values even as the loss decreases to zero—helps explain the stability and convergence properties observed in overparameterized networks.

The NTK framework builds a rigorous bridge between deep learning and classical statistical learning theory. At initialization, an infinitely wide network defines a Gaussian Process (GP) with the Neural Network Gaussian Process (NNGP) kernel Neal (1996); Lee et al. (2018), characterizing the function-space distribution before any training. During training, the dynamics are captured by the NTK, which describes how the function evolves under gradient descent. Both kernels become deterministic in the infinite-width limit, enabling exact characterization of learning dynamics through tools from functional analysis and kernel theory.

This report develops the mathematical foundations and intuitions underlying the NTK. We begin in Section 2 with a self-contained treatment of RKHS theory, covering both scalar-valued and vector-valued kernels Alvarez et al. (2012). The extension to vector-valued functions is essential for understanding multi-output networks, as the NTK for an m-output network is precisely a matrix-valued kernel encoding both input-space relationships and output correlations. Section 3 reviews Gaussian Processes and their connection to neural networks at initialization, establishing the NNGP kernel as the starting point for understanding training dynamics. In Section 4, we derive the NTK for fully connected networks (borrowing from Weng (2022)), show its convergence to a deterministic limit as width grows, and analyze the resulting linear dynamics that govern lazy training.

The NTK perspective offers both insights and limitations. It successfully explains why gradient descent finds global optima (the kernel induces a convex problem in function space), why random initialization can work well (it defines a meaningful RKHS), and provides exact training dynamics for wide networks. However, NTK theory also highlights a fundamental tension: while it proves that infinite-width networks behave as kernel machines, empirical evidence shows that finite-width networks consistently outperform kernel methods on complex tasks. This gap reveals that practical deep learning derives its power from operating *outside* the lazy regime, where finite width enables genuine feature learning rather than mere linear regression in a fixed feature space.

By establishing NTK as a rigorous baseline—the simplest regime amenable to complete theoretical analysis—we gain a reference point for understanding what makes deep learning "deep." The deviations from NTK behavior at realistic widths, particularly the emergence of hierarchical feature learning and adaptive representations, illuminate the mechanisms underlying the remarkable success of modern neural networks. This report aims to provide the mathematical tools and conceptual frameworks needed to navigate this landscape, connecting classical kernel theory with contemporary deep learning through the unifying lens of the Neural Tangent Kernel.

2 Reproducing Kernel Hilbert Spaces (RKHS)

2.1 A Brief Overview

Kernel methods operate in large (possibly infinite-dimensional) feature spaces without explicitly computing features. The key mathematical structure is the *Reproducing Kernel Hilbert Space* (RKHS)—a Hilbert space $\mathcal{H}$ of functions where point evaluation is continuous. This seemingly simple requirement has profound consequences: for every point $x \in \mathcal{X}$, there exists a unique function $k_x \in \mathcal{H}$ such that $f(x) = \langle f, k_x \rangle_{\mathcal{H}}$ for all $f \in \mathcal{H}$. The *reproducing kernel* $K(x, y) = \langle k_x, k_y \rangle_{\mathcal{H}}$ reproduces function values through inner products.

The celebrated Moore-Aronszajn theorem establishes a bijection: every positive definite kernel K defines a unique RKHS $\mathcal{H}_K$, and conversely. This means choosing a kernel K automatically specifies an entire function space with its geometry. The representer theorem completes the picture for learning: solutions to regularized empirical risk minimization in $\mathcal{H}_K$ always have the finite-dimensional form $f^*(x) = \sum_{i=1}^{n} \alpha_i K(x, x_i)$, reducing infinite-dimensional optimization to a tractable n-dimensional problem.

For neural networks with multiple outputs, we need *vector-valued RKHS* with matrix-valued kernels $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^{m \times m}$. The theory generalizes naturally: the kernel encodes both input-space relationships and output correlations. Common constructions include separable kernels $K(x, x') = k(x, x')\mathbf{B}$ and the Linear Model of Coregionalization $K(x, x') = \sum_q k_q(x, x')\mathbf{B}_q$, which enables multi-task learning by sharing information across related outputs.

The NTK for an m-output network is precisely a matrix-valued kernel—understanding vector-valued RKHS is understanding how network outputs interact during training.

Kernel methods operate in large feature spaces without explicitly computing features. The mathematical structure enabling this is the *Reproducing Kernel Hilbert Space* (RKHS)—a Hilbert space $\mathcal{H}$ of functions where point evaluation is continuous. This implies: for every $x \in \mathcal{X}$, there exists a unique function $k_x \in \mathcal{H}$ such that the evaluation functional at x denoted as $E_x \in \mathcal{H}^*$ (dual space of $\mathcal{H}$) and $E_x : \mathcal{H} \to \mathbb{R}$ is $E_x(f) = f(x) = \langle f, k_x \rangle_{\mathcal{H}}$ such that $E_x \in \mathcal{H}^*$ is identified with $k_x \in \mathcal{H}$ (by the Riesz representation theorem) . The kernel $K(x, y) = \langle k_x, k_y \rangle_{\mathcal{H}}$ reproduces function values through inner products because $f(x)$

The Moore–Aronszajn theorem completes the picture: every positive definite kernel defines a unique RKHS (this is done by building a Hilbert space $\mathcal{H}$ by partial application of the argument on the kernel K to get $K(x, \cdot)$, creating a vector space by linear combination and then completing it by including all its Cauchy limits. Choosing a kernel K specifies the entire function space and its geometry.

2.2 SCALAR-VALUED RKHS

2.2.1 DEFINITIONS

Definition 1 (RKHS). *A Hilbert space $\mathcal{H}$ of functions $f : \mathcal{X} \to \mathbb{R}$ is an RKHS if for every $x \in \mathcal{X}$, the evaluation functional $E_x(f) = f(x)$ is bounded.*

By Riesz representation, there exists unique $k_x \in \mathcal{H}$ with $f(x) = \langle f, k_x \rangle_{\mathcal{H}}$.

Definition 2 (Reproducing Kernel). *$K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ defined by $K(x, y) = \langle k_y, k_x \rangle_{\mathcal{H}}$ is the* reproducing kernel.

Properties: (1) $f(x) = \langle f, K(\cdot, x) \rangle$, (2) $K(x, y) = K(y, x)$, (3) $\|E_x\| = \sqrt{K(x, x)}$, (4) K determines $\mathcal{H}$ uniquely.

2.2.2 MOORE-ARONSZAJN THEOREM [$\sim$ 1950]

Definition 3 (Positive Definite Kernel). *$K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is* positive definite *if $\sum_{i,j} c_i c_j K(x_i, x_j) \geq 0$ for all finite $\{x_i\} \subset \mathcal{X}$, $\{c_i\} \subset \mathbb{R}$.*

Theorem 1 (Moore-Aronszajn). *Positive definite kernels on $\mathcal{X}$ bijectively correspond to RKHS on $\mathcal{X}$.*

Construction of $\mathcal{H}_K$: Given positive definite K, define $k_y := K(\cdot, y)$ such that $k_y(x) = K(x, y)$ and let $W = \text{span}\{k_y\}$. Define $\langle \sum_j \alpha_j k_{y_j}, \sum_i \beta_i k_{x_i} \rangle_W := \sum_{i,j} \alpha_j \beta_i K(x_i, y_j)$. This is well-defined, satisfies $\langle f, k_x \rangle_W = f(x)$, and $\langle f, f \rangle_W \geq 0$ with equality iff $f = 0$. Complete W to obtain $\mathcal{H}_K$.

Feature map: $\phi(x) = k_x$ gives $K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}_K}$ (the kernel trick).

Examples: Linear $K(x, x') = x^T x'$, polynomial $(x^T x' + c)^p$, Gaussian $\exp(-\|x - x'\|^2 / 2\sigma^2)$.

2.2.3 REPRESENTER THEOREM

Theorem 2 (Kernel via Basis). *If $\{e_s\}$ is an orthonormal basis for $\mathcal{H}$, then $K(x, y) = \sum_s e_s(x) e_s(y)$.*

Theorem 3 (Representer Theorem). *For $\min_{f \in \mathcal{H}_K} \left[\sum_{i=1}^n L(y_i, f(x_i)) + \lambda \|f\|^2 \right]$, any minimizer has form $f^*(x) = \sum_{i=1}^n \alpha_i K(x, x_i)$.*

Proof: Write $f = f_\parallel + f_\perp$ where $f_\parallel \in \text{span}\{k_{x_i}\}$. Since $f_\perp(x_i) = 0$, it doesn't affect loss but increases $\|f\|^2$, so optimal $f_\perp = 0$. □

2.3 VECTOR-VALUED RKHS

Neural networks with multiple outputs (multi-class classification, multi-task learning) require extending RKHS theory to vector-valued functions. Nearly everything from the scalar case generalizes naturally more or less . Alvarez et al. (2012); Jorgensen & Tian (2024).

2.3.1 MATRIX-VALUED KERNELS

A *matrix-valued kernel* $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^{m \times m}$ assigns an $m \times m$ matrix to each pair of inputs. The matrix $K(x, x')$ encodes both how inputs x and x' relate, and how the m output dimensions correlate with each other.

Positive Definiteness. We require $\sum_{i,j} c_i^T K(x_i, x_j) c_j \geq 0$ for all finite $\{x_i\} \subset \mathcal{X}$ and $\{c_i\} \subset \mathbb{R}^m$. Equivalently, the $nm \times nm$ block matrix $\mathbf{K} = [K(x_i, x_j)]_{i,j}$ must be positive semidefinite.

Key Constructions:

1. **Separable:** $K(x, x') = k(x, x')\mathbf{B}$ where k is a scalar kernel and $\mathbf{B} \in \mathbb{R}^{m\times m}$ encodes output correlations. Simple but assumes all outputs share the same input-space structure.
2. **Linear Model of Coregionalization (LMC):** $K(x, x') = \sum_{q=1}^{Q} k_q(x, x')\mathbf{B}_q$. Different kernels k_q capture different scales; matrices $\mathbf{B}_q$ encode output relationships at each scale. This is the workhorse for multi-task learning. It lets related tasks help each other: if output 1 (say, French translation) correlates with output 2 (Spanish translation) via $\mathbf{B}_q$, then data for one task informs the other.
3. **Block diagonal:** $K_{ij}(x, x') = \delta_{ij}k_i(x, x')$. Independent outputs–no information sharing.

2.3.2 Vector-Valued Reproducing Property

For $f : \mathcal{X} \to \mathbb{R}^m$ in a vector-valued RKHS $\mathcal{H}$, the reproducing property is

$$\langle f, K_x v\rangle_{\mathcal{H}} = v^T f(x)$$

for all $v \in \mathbb{R}^m$. Setting $v = e_j$ gives $\langle f, K_x e_j\rangle = f_j(x)$—each component's evaluation funcitonal (E_x^j) is continuous.

2.3.3 Representer Theorem

Theorem 4 (Vector-Valued Representer Theorem)**.** *For* $\min_{f\in\mathcal{H}_K}\left[\sum_{i=1}^{n} L(y_i, f(x_i)) + \lambda\|f\|^2\right]$ *with* $y_i \in \mathbb{R}^m$*, the minimizer has form*

$$f^*(x) = \sum_{i=1}^{n} K(x, x_i)\alpha_i$$

for some $\{\alpha_i\}_{i=1}^n \subset \mathbb{R}^m$.

Same story as the scalar case: optimize over the finite-dimensional coefficients $\{\alpha_i\}$ using the kernel matrix, despite $\mathcal{H}_K$ being infinite-dimensional.

Computation. For separable $K(x, x') = k(x, x')\mathbf{B}$, solve $(\mathbf{G} \otimes \mathbf{B} + \lambda\mathbf{I})\boldsymbol{\alpha} = \mathbf{y}$ where $\mathbf{G}_{ij} = k(x_i, x_j)$. Eigendecompose $\mathbf{B}$ to reduce this to m independent $n \times n$ problems—efficient!

Connection to Neural Networks. The NTK for a network with m outputs is exactly a matrix-valued kernel $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^{m\times m}$. Understanding vector-valued RKHS is understanding how neural network outputs interact during training.

3 Gaussian Processes

Gaussian processes (GPs) provide a powerful probabilistic framework for function learning. They appear naturally as the infinite-width limit of neural networks and form the foundation for understanding the Neural Network Gaussian Process (NNGP) kernel.

3.1 Definition and Basic Properties

Definition 4 (Gaussian Process)**.** *A* Gaussian process *is a collection of random variables* $\{f(x) : x \in \mathcal{X}\}$ *such that any finite subset* $(f(x_1), \ldots, f(x_n))$ *has a joint Gaussian distribution.*

A GP is completely specified by its mean function $m : \mathcal{X} \to \mathbb{R}$ and covariance function (kernel) $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$:

$$m(x) = \mathbb{E}[f(x)]$$
$$k(x, x') = \mathbb{E}[(f(x) - m(x))(f(x') - m(x'))]$$

We write $f \sim \mathcal{GP}(m, k)$. For any finite collection $\mathbf{x} = (x_1, \ldots, x_n)$:

$$\mathbf{f} := (f(x_1), \ldots, f(x_n))^T \sim \mathcal{N}(\mathbf{m}, \mathbf{K})$$

where $\mathbf{m}_i = m(x_i)$ and $\mathbf{K}_{ij} = k(x_i, x_j)$.

Why "process"? The term emphasizes that we're defining a probability distribution over *functions*, not just vectors. A GP assigns probabilities to function values $f(x)$ at every point $x \in \mathcal{X}$ simultaneously.

3.2 Intuition: Functions as Infinite-Dimensional Gaussians

Finite-dimensional analogy. A multivariate Gaussian $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ in $\mathbb{R}^n$ is completely specified by:

- Mean vector $\boldsymbol{\mu} \in \mathbb{R}^n$ (where the distribution is centered)
- Covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$ (how dimensions correlate)

Infinite-dimensional extension. A GP extends this to functions $f : \mathcal{X} \to \mathbb{R}$:

- Mean function $m : \mathcal{X} \to \mathbb{R}$ replaces the mean vector
- Covariance kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ replaces the covariance matrix

The kernel $k(x, x')$ determines how function values at different points correlate. Large $k(x, x')$ means $f(x)$ and $f(x')$ tend to be similar; small $k(x, x')$ means they vary independently.

Sampling interpretation. Drawing a sample from a GP means:

1. Choose finitely many evaluation points $x_1, \ldots, x_n$
2. Sample $\mathbf{f} \sim \mathcal{N}(\mathbf{m}, \mathbf{K})$ where $\mathbf{K}_{ij} = k(x_i, x_j)$
3. Interpolate to get a continuous function $f(x)$

Different kernels give different function characteristics (smooth, rough, periodic, etc.).

3.3 The Gaussian Process Prior

GPs are most commonly used with a zero mean prior: $f \sim \mathcal{GP}(0, k)$. The kernel encodes our beliefs about the function:

Squared Exponential (RBF) kernel:

$$k_{\text{SE}}(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right)$$

- Hyperparameter ℓ: length scale (how quickly correlations decay)
- Hyperparameter σ^2: output variance (vertical scale)
- Produces infinitely differentiable (smooth) functions
- $k(x, x') \to 0$ exponentially fast as $\|x - x'\| \to \infty$

Matérn kernel:

$$k_\nu(x, x') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\|x - x'\|}{\ell}\right)^\nu K_\nu \left(\frac{\sqrt{2\nu}\|x - x'\|}{\ell}\right)$$

- Parameter ν controls smoothness: $\nu = 1/2$ gives rough paths, $\nu \to \infty$ gives RBF
- More flexible than RBF for modeling real data

Linear kernel:

$$k_{\text{lin}}(x, x') = x^T x'$$

Corresponds to Bayesian linear regression.

3.4 GAUSSIAN PROCESS REGRESSION

Given training data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ with $y_i = f(x_i) + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$, and prior $f \sim \mathcal{GP}(0, k)$, the posterior at test point x_* is:

Theorem 5 (GP Posterior)**.** *The posterior distribution is Gaussian:*

$$f(x_*) \mid \mathcal{D} \sim \mathcal{N}(\mu_*, \sigma_*^2)$$

where

$$\mu_* = \mathbf{k}_*^T(\mathbf{K} + \sigma_n^2\mathbf{I})^{-1}\mathbf{y}$$
$$\sigma_*^2 = k(x_*, x_*) - \mathbf{k}_*^T(\mathbf{K} + \sigma_n^2\mathbf{I})^{-1}\mathbf{k}_*$$

with $\mathbf{k}_* = (k(x_*, x_1), \ldots, k(x_*, x_n))^T$ *and* $\mathbf{K}_{ij} = k(x_i, x_j)$.

Interpretation:

- μ_* is the posterior mean prediction (weighted average of training outputs)
- σ_*^2 is the posterior variance (uncertainty in prediction)
- At training points: $\mu(x_i) \approx y_i$ and $\sigma(x_i) \approx 0$ (low uncertainty)
- Far from data: $\mu(x) \to 0$ (prior mean) and $\sigma(x) \to k(x, x)$ (prior uncertainty)

Connection to kernel ridge regression. The posterior mean μ_* is identical to kernel ridge regression with regularization $\lambda = \sigma_n^2$:

$$\mu_* = \sum_{i=1}^n \alpha_i k(x_*, x_i), \quad \boldsymbol{\alpha} = (\mathbf{K} + \sigma_n^2\mathbf{I})^{-1}\mathbf{y}$$

GP regression adds uncertainty quantification via σ_*^2.

3.5 RELATIONSHIP TO RKHS

There's a deep connection between GPs and RKHS:

Theorem 6 (GP-RKHS Connection)**.** *Let $f \sim \mathcal{GP}(0, k)$ where k is a positive definite kernel. Then:*

1. *Sample paths f are* not *in $\mathcal{H}_k$ (the RKHS of k) almost surely*
2. *However, $\mathbb{E}[\|f\|_{\mathcal{H}_k}^2] < \infty$ (functions have finite expected RKHS norm)*
3. *The posterior mean μ_* is in $\mathcal{H}_k$ with $\|\mu_*\|_{\mathcal{H}_k}^2 = \mathbf{y}^T(\mathbf{K} + \sigma_n^2\mathbf{I})^{-1}\mathbf{y}$*

Intuition: Think of the RKHS $\mathcal{H}_k$ as the "smooth core" of functions favored by kernel k. GP samples are rougher than RKHS functions (they have some randomness), but their posterior mean—what we actually predict with—lives in the RKHS.

3.6 NEURAL NETWORKS AS GAUSSIAN PROCESSES

One of the most important results connecting NNs and GPs:

Theorem 7 (Neal (1996); Lee et al. (2018))**.** *Consider a fully connected neural network with one hidden layer of width m:*

$$f(x) = \frac{1}{\sqrt{m}} \sum_{i=1}^m v_i \sigma(w_i^T x + b_i)$$

with i.i.d. Gaussian initialization $v_i, w_{ij}, b_i \sim \mathcal{N}(0, 1)$.

As $m \to \infty$, the network function f converges in distribution to a Gaussian process:

$$f \sim \mathcal{GP}(0, k_{NNGP}^{(1)})$$

where

$$k_{NNGP}^{(1)}(x, x') = \mathbb{E}_{w,b\sim\mathcal{N}(0,1)}[\sigma(w^T x + b)\sigma(w^T x' + b)]$$

Why this happens: By the Central Limit Theorem, the sum $\sum_{i=1}^{m} v_i \sigma(w_i^T x + b_i)$ is a sum of i.i.d. random variables (for fixed x). As $m \to \infty$, this converges to a Gaussian. For any finite set of inputs $(x_1, \ldots, x_n)$, we get a multivariate Gaussian—hence a GP.

Extension to deep networks. For L-layer networks, the limiting kernel can be computed recursively:

$$\Sigma^{(0)}(x, x') = x^T x'$$
$$\Sigma^{(\ell+1)}(x, x') = \mathbb{E}_{(u,v)\sim\mathcal{N}(0,C^{(\ell)})}[\sigma(u)\sigma(v)]$$

where $C^{(\ell)}$ is the 2×2 covariance $\begin{pmatrix} \Sigma^{(\ell)}(x,x) & \Sigma^{(\ell)}(x,x') \\ \Sigma^{(\ell)}(x',x) & \Sigma^{(\ell)}(x',x') \end{pmatrix}$.

This is the NNGP kernel at depth ℓ.

3.7 Key Takeaways

1. **GPs are distributions over functions.** They extend multivariate Gaussians from finite to infinite dimensions, specified by mean and covariance functions.
2. **The kernel determines everything.** Choice of kernel $k(x, x')$ encodes assumptions about smoothness, length scales, and correlations.
3. **GP regression = kernel methods + uncertainty.** The posterior mean equals kernel ridge regression; the posterior variance quantifies uncertainty.
4. **GPs and RKHS are complementary.** GPs provide a probabilistic view (Bayesian inference), RKHS provide a functional analytic view (optimization). Same kernel, different perspectives.
5. **Infinite-width NNs are GPs.** At initialization, wide networks converge to GPs with the NNGP kernel. This explains why random initialization can work well.
6. **Limitation: No feature learning.** GPs (and NNGPs) have fixed kernels that don't adapt during training. Real neural networks learn features by evolving their representations—this happens at finite width outside the GP limit.
7. **Connection to NTK.** The NNGP describes the *function* at initialization. The NTK describes how the function *evolves during training*. Both become deterministic in the infinite-width limit.

3.8 Computational Considerations

Cost: GP regression requires:

- Computing $\mathbf{K}$: $O(n^2)$ kernel evaluations
- Inverting $\mathbf{K} + \sigma_n^2 \mathbf{I}$: $O(n^3)$ time
- Prediction at n_* test points: $O(n \cdot n_*)$ time

This cubic scaling limits GPs to datasets with $n \lesssim 10{,}000$.

Sparse approximations: Use inducing points, variational inference, or random features to scale to larger datasets. These trade exact inference for computational efficiency.

Relation to neural networks: Neural networks avoid the $O(n^3)$ bottleneck by using gradient-based optimization on parameters instead of direct kernel inversion. At infinite width, they *are* GPs, but their finite-width parametric form makes them tractable for large-scale learning.

4 Neural Tangent Kernel (NTK)

We've seen that RKHS theory elegantly describes kernel methods. Now comes the surprise: infinitely wide neural networks trained by gradient descent *are* kernel methods. The Neural Tangent Kernel makes this connection precise.

But first, let's understand what "kernel gradient descent" means in a familiar setting.

4.1 WARM-UP: KERNEL GRADIENT DESCENT IN SVMS

Consider training a support vector machine (SVM) for binary classification with data $\{(x_i, y_i)\}_{i=1}^n$ where $y_i \in \{-1, +1\}$. We seek a function $f \in \mathcal{H}_K$ minimizing the hinge loss:

$$\mathcal{L}(f) = \sum_{i=1}^{n} \max(0, 1 - y_i f(x_i)) + \lambda \|f\|_{\mathcal{H}_K}^2$$

Instead of solving this directly, imagine using gradient flow in the RKHS. At time t, we have some $f_t \in \mathcal{H}_K$, and we update:

$$\frac{df_t}{dt} = -\nabla_{\mathcal{H}_K} \mathcal{L}(f_t)$$

What does this gradient look like? The functional derivative gives:

$$\nabla_{\mathcal{H}_K} \mathcal{L}(f_t) = -\sum_{i: y_i f_t(x_i) < 1} y_i K(\cdot, x_i) + 2\lambda f_t$$

This is the *kernel gradient*—a linear combination of kernel functions $K(\cdot, x_i) \in \mathcal{H}_K$ (see Appendix 2) centered at the training points!

The dynamics become:

$$\frac{df_t(x)}{dt} = -\sum_{i: y_i f_t(x_i) < 1} y_i K(x, x_i) + 2\lambda f_t(x)$$

Notice the structure: updates at training point x_i affect the value at x through the kernel $K(x, x_i)$. Points with large $K(x, x_i)$ (similar to x_i in the feature space) receive strong updates.

Key insight: Even though $\mathcal{H}_K$ is infinite-dimensional, the gradient flow stays in $\text{span}\{K(\cdot, x_i)\}_{i=1}^n$—a finite-dimensional subspace. Why? The representer theorem (Appendix 2) guarantees that solutions to regularized risk minimization always have the form $f^* = \sum_i \alpha_i K(\cdot, x_i)$. This is kernel gradient descent: optimization in function space that reduces to finite-dimensional linear algebra via the kernel trick.

Now let's see this same phenomenon in neural networks.

4.2 SETUP: NETWORKS AS PARAMETERIZED FUNCTIONS

Consider a fully connected network with L layers, widths $(n_0, n_1, \ldots, n_L)$, and smooth activation $\sigma : \mathbb{R} \to \mathbb{R}$. We'll think of this network as a map from parameters to functions:

$$F : \theta \mapsto f_\theta$$

where $\theta = \{W^{(\ell)}, b^{(\ell)}\}_{\ell=1}^L$ and $f_\theta : \mathbb{R}^{n_0} \to \mathbb{R}^{n_L}$.

Forward pass with NTK scaling. To make the infinite-width limit well-behaved, we use:

$$\begin{aligned}
h^{(0)}(x) &= x \\
h^{(\ell)}(x) &= \sigma\left(\frac{1}{\sqrt{n_{\ell-1}}} W^{(\ell)} h^{(\ell-1)}(x) + b^{(\ell)}\right), \quad \ell = 1, \ldots, L-1 \\
f_\theta(x) &= \frac{1}{\sqrt{n_{L-1}}} W^{(L)} h^{(L-1)}(x) + b^{(L)}
\end{aligned}$$

The $1/\sqrt{n_{\ell-1}}$ factors prevent activations from exploding as width grows. Initialize $W_{ij}^{(\ell)}, b_i^{(\ell)} \sim \mathcal{N}(0, 1)$ i.i.d.

Training dynamics. We minimize loss $\mathcal{L}(\theta) = \frac{1}{2} \sum_{i=1}^n \|f_\theta(x_i) - y_i\|^2$ via gradient flow:

$$\frac{d\theta}{dt} = -\nabla_\theta \mathcal{L}(\theta).$$

The key question: how does $f_\theta(x)$ evolve during training?

4.3 DERIVATION OF THE NTK

Step 1: Linearize the network. By the chain rule:

$$\frac{df_\theta(x)}{dt} = \sum_p \frac{\partial f_\theta(x)}{\partial \theta_p}\frac{d\theta_p}{dt} = -\sum_p \frac{\partial f_\theta(x)}{\partial \theta_p}\frac{\partial \mathcal{L}}{\partial \theta_p}$$

Step 2: Express via training points. Since $\mathcal{L}(\theta) = \frac{1}{2}\sum_i \|f_\theta(x_i) - y_i\|^2$:

$$\frac{\partial \mathcal{L}}{\partial \theta_p} = \sum_{i=1}^{n}\sum_{k=1}^{n_L} \frac{\partial f_{\theta,k}(x_i)}{\partial \theta_p}(f_{\theta,k}(x_i) - y_{i,k})$$

Substituting back:

$$\frac{df_{\theta,j}(x)}{dt} = -\sum_{i=1}^{n}\sum_{k=1}^{n_L}\left(\sum_p \frac{\partial f_{\theta,j}(x)}{\partial \theta_p}\frac{\partial f_{\theta,k}(x_i)}{\partial \theta_p}\right)(f_{\theta,k}(x_i) - y_{i,k})$$

Step 3: Define the NTK. The inner sum is precisely a kernel:

$$\Theta_{jk}(x, x'; \theta) := \sum_p \frac{\partial f_{\theta,j}(x)}{\partial \theta_p}\frac{\partial f_{\theta,k}(x')}{\partial \theta_p} = \langle \nabla_\theta f_{\theta,j}(x), \nabla_\theta f_{\theta,k}(x')\rangle \tag{1}$$

This is the *Neural Tangent Kernel*—an $n_L \times n_L$ matrix-valued kernel measuring how gradient updates at x' affect the output at x.

Step 4: Dynamics as kernel gradient flow. Define $u(t) = [f_\theta(x_1)^T, \ldots, f_\theta(x_n)^T]^T \in \mathbb{R}^{n\cdot n_L}$ and the gram matrix $\mathbf{K}(t)_{ij} = \Theta(x_i, x_j; \theta(t))$. Then:

$$\frac{du(t)}{dt} = -\mathbf{K}(t)(u(t) - y)$$

The network follows kernel gradient descent! Just like in the SVM example, updates at training point x_i affect predictions at x through $\Theta(x, x_i)$. But there's a catch: $\mathbf{K}(t)$ depends on time because Θ depends on $\theta(t)$.

4.4 THE INFINITE-WIDTH LIMIT

Here's where the theory becomes beautiful. As all hidden layer widths $n_1, \ldots, n_{L-1} \to \infty$, two things happen:

Theorem 8 (Jacot et al., 2018)**.** *At initialization, f_θ converges to a Gaussian process with covariance $\Sigma^{(L)}$, and Θ converges in probability to a deterministic kernel Θ^∞ (both independent of θ). Moreover, during training, $\Theta(\cdot, \cdot; \theta(t)) \to \Theta^\infty$ stays constant.*

Why does this happen?

At initialization: Consider the last layer. We have $f_{\theta,j}(x) = \frac{1}{\sqrt{n_{L-1}}}\sum_{i=1}^{n_{L-1}} W_{ji}^{(L)}\sigma(z_i(x)) + b_j^{(L)}$ where $z_i(x)$ are the pre-activations. By the law of large numbers, as $n_{L-1} \to \infty$:

$$\frac{1}{n_{L-1}}\sum_{i=1}^{n_{L-1}} \sigma(z_i(x))\sigma(z_i(x')) \to \mathbb{E}[\sigma(Z(x))\sigma(Z(x'))]$$

where Z is the limiting Gaussian process for the previous layer. This gives Gaussian outputs with deterministic covariance.

During training: Each parameter moves infinitesimally (there are infinitely many sharing the work), so $\nabla_\theta f_\theta(x)$ barely changes from initialization. Thus $\Theta(\theta(t)) \approx \Theta(\theta_0) \to \Theta^\infty$ for all t.

4.5 COMPUTING THE INFINITE-WIDTH NTK

The limiting kernels are defined recursively. First, the *Neural Network Gaussian Process (NNGP)* kernel:

$$\Sigma^{(1)}(x, x') = x^T x' + 1$$
$$\Sigma^{(\ell+1)}(x, x') = \mathbb{E}_{(u,v)\sim\mathcal{N}(0,C^{(\ell)})}[\sigma(u)\sigma(v)] + 1$$

where $C^{(\ell)}$ is the 2×2 covariance matrix $\begin{pmatrix} \Sigma^{(\ell)}(x,x) & \Sigma^{(\ell)}(x,x') \\ \Sigma^{(\ell)}(x',x) & \Sigma^{(\ell)}(x',x') \end{pmatrix}$.

The NTK satisfies:

$$\Theta^{(1)}(x, x') = \Sigma^{(1)}(x, x')$$
$$\Theta^{(\ell+1)}(x, x') = \Theta^{(\ell)}(x, x') \cdot \dot{\Sigma}^{(\ell+1)}(x, x') + \Sigma^{(\ell+1)}(x, x')$$

where $\dot{\Sigma}^{(\ell+1)}(x, x') = \mathbb{E}_{(u,v)\sim\mathcal{N}(0,C^{(\ell)})}[\sigma'(u)\sigma'(v)]$.

Interpretation: The recursion has two terms:

- $\Sigma^{(\ell+1)}$: contribution from training the last layer (treating lower layers as frozen features)
- $\Theta^{(\ell)} \cdot \dot{\Sigma}^{(\ell+1)}$: contribution from training all lower layers (feature learning, weighted by how much activations change)

For ReLU, these expectations can be computed in closed form using Gaussian concentration on cones.

4.6 TRAINING DYNAMICS IN THE LAZY REGIME

With constant kernel $\mathbf{K}^\infty$, we have a linear ODE:

$$\frac{du(t)}{dt} = -\mathbf{K}^\infty(u(t) - y)$$

Solution: If $\mathbf{K}^\infty$ is positive definite (invertible), then:

$$u(t) = y + e^{-\mathbf{K}^\infty t}(u(0) - y)$$

Convergence is exponential with rate determined by eigenvalues of $\mathbf{K}^\infty$. Larger eigenvalues mean faster convergence along those directions.

Connection to kernel PCA. The dynamics converges fastest along directions with large eigenvalues of $\mathbf{K}^\infty$—these are the kernel principal components of the training data. This suggests early stopping: train just long enough to fit the major components (signal) without fitting the minor ones (noise).

The lazy regime. This is characterized by:

$$\|\theta(t) - \theta(0)\| \ll \|\theta(0)\| \quad \text{(parameters barely move)}$$
$$\|u(t) - y\| \to 0 \quad \text{(yet training error vanishes)}$$

The network is "lazy"—it doesn't learn features, just does linear regression in a fixed, random feature space defined by $\nabla_\theta f_{\theta_0}(x)$.

When does it occur? The lazy regime holds when:

- Width is huge: $\min(n_1, \ldots, n_{L-1}) \gg n$ (extreme overparameterization)
- Initialization is standard: $\mathcal{N}(0, 1/n_\ell)$ (NTK scaling)
- Learning rate is moderate (too large breaks the linearization)

4.7 CONNECTION TO VECTOR-VALUED RKHS

The limiting NTK $\Theta^\infty : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^{n_L \times n_L}$ is a positive definite matrix-valued kernel (under mild conditions). By the Moore-Aronszajn theorem, it defines a unique vector-valued RKHS $\mathcal{H}_{\Theta^\infty}$ of functions $f : \mathcal{X} \to \mathbb{R}^{n_L}$.

Training an infinite-width network is equivalent to:

1. Starting at $f_0 \sim \mathcal{GP}(0, \Theta^\infty)$
2. Performing kernel gradient descent in $\mathcal{H}_{\Theta^\infty}$
3. Converging to the minimum-norm interpolant: $\min_{f \in \mathcal{H}_{\Theta^\infty}} \|f\|_{\mathcal{H}}$ subject to $f(x_i) = y_i$

This is exactly kernel ridge regression as $\lambda \to 0$.

4.8 WHAT NTK EXPLAINS—AND WHAT IT DOESN'T

Successes:

- Why gradient descent converges globally (positive definite kernel $\Rightarrow$ convex problem in function space)
- Why random initialization works (defines a good RKHS)
- Exact training dynamics for wide networks (linear ODE)

Limitations:

- *Generalization:* NTK doesn't explain why deep networks generalize. Kernel methods can memorize noise.
- *Feature learning:* Real networks at realistic widths learn hierarchical representations. NTK assumes frozen features.
- *Expressivity:* Finite-width networks outperform kernel methods on vision and language. The power comes from escaping the lazy regime.

The fundamental tension. NTK says infinite-width networks are kernel machines. But kernel methods can't match deep learning performance. Resolution: practical deep learning operates *outside* the lazy regime, where width is large but finite, and the network genuinely learns features.

NTK provides a rigorous baseline—the simplest regime we can fully analyze. Understanding how realistic networks deviate from NTK reveals where the "deep" in deep learning comes from.

With constant kernel $\mathbf{K}^\infty$, we have a linear ODE:

$$\frac{du(t)}{dt} = -\mathbf{K}^\infty (u(t) - y)$$

Solution: If $\mathbf{K}^\infty$ is positive definite (invertible), then:

$$u(t) = y + e^{-\mathbf{K}^\infty t}(u(0) - y)$$

Convergence is exponential with rate determined by eigenvalues of $\mathbf{K}^\infty$. Larger eigenvalues mean faster convergence along those directions.

Connection to kernel PCA. The dynamics converges fastest along directions with large eigenvalues of $\mathbf{K}^\infty$—these are the kernel principal components of the training data. This suggests early stopping: train just long enough to fit the major components (signal) without fitting the minor ones (noise).

The lazy regime. This is characterized by:

$$\|\theta(t) - \theta(0)\| \ll \|\theta(0)\| \quad \text{(parameters barely move)}$$
$$\|u(t) - y\| \to 0 \quad \text{(yet training error vanishes)}$$

The network is "lazy"—it doesn't learn features, just does linear regression in a fixed, random feature space defined by $\nabla_\theta f_{\theta_0}(x)$.

When does it occur? The lazy regime holds when:

- Width is huge: $\min(n_1, \ldots, n_{L-1}) \gg n$ (extreme overparameterization)
- Initialization is standard: $\mathcal{N}(0, 1/n_\ell)$ (NTK scaling)
- Learning rate is moderate (too large breaks the linearization)

4.9 Connection to Vector-Valued RKHS

The limiting NTK $\Theta^\infty : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^{n_L \times n_L}$ is a positive definite matrix-valued kernel (under mild conditions). By the Moore-Aronszajn theorem, it defines a unique vector-valued RKHS $\mathcal{H}_{\Theta^\infty}$ of functions $f : \mathcal{X} \to \mathbb{R}^{n_L}$.

Training an infinite-width network is equivalent to:

1. Starting at $f_0 \sim \mathcal{GP}(0, \Theta^\infty)$ (see Appendix 3 for a review of Gaussian Processes)
2. Performing kernel gradient descent in $\mathcal{H}_{\Theta^\infty}$
3. Converging to the minimum-norm interpolant: $\min_{f \in \mathcal{H}_{\Theta^\infty}} \|f\|_{\mathcal{H}}$ subject to $f(x_i) = y_i$

This is exactly kernel ridge regression as $\lambda \to 0$.

4.10 What NTK Explains—and What It Doesn't

Successes:

- Why gradient descent converges globally (positive definite kernel $\Rightarrow$ convex problem in function space)
- Why random initialization works (defines a good RKHS)
- Exact training dynamics for wide networks (linear ODE)

Limitations:

- *Generalization:* NTK doesn't explain why deep networks generalize. Kernel methods can memorize noise.
- *Feature learning:* Real networks at realistic widths learn hierarchical representations. NTK assumes frozen features.
- *Expressivity:* Finite-width networks outperform kernel methods on vision and language. The power comes from escaping the lazy regime.

The fundamental tension. NTK says infinite-width networks are kernel machines. But kernel methods can't match deep learning performance. Resolution: practical deep learning operates *outside* the lazy regime, where width is large but finite, and the network genuinely learns features.

NTK provides a rigorous baseline—the simplest regime we can fully analyze. Understanding how realistic networks deviate from NTK reveals where the "deep" in deep learning comes from.

4.10.1 Applications and Connections

Multi-Task Learning. Vector-valued kernels provide a principled framework for multi-task learning, where the goal is to simultaneously learn m related prediction tasks. The coregionalization matrix $\mathbf{B}$ encodes task relatedness: large off-diagonal entries B_{ij} indicate that tasks i and j are strongly correlated and should share information. This is formalized through the shared RKHS structure, which pools data across tasks weighted by their correlations.

Multi-Class Classification. In multi-class classification with m classes, we can use vector-valued functions $f : \mathcal{X} \to \mathbb{R}^m$ where $f_j(x)$ represents the score for class j. The prediction is then $\arg\max_j f_j(x)$. Vector-valued kernels allow the classifier to exploit correlations between classes (e.g., hierarchical relationships in label space).

Connection to Gaussian Processes. From a Bayesian perspective, a vector-valued kernel K serves as the covariance function for a *multi-output Gaussian Process*. If $\mathbf{f} = (f_1, \ldots, f_m)$ is a random

vector-valued function with $f_i \sim \text{GP}(0, k_i)$ independently, this corresponds to a block-diagonal kernel. More generally, dependent Gaussian processes with kernel K model correlations between outputs, enabling probabilistic multi-task learning with uncertainty quantification.

Transfer Learning. When tasks have different amounts of training data, vector-valued kernels enable transfer learning by allowing data-rich tasks to inform data-poor tasks through the off-diagonal entries of the coregionalization matrix. This is particularly valuable in domains like multi-lingual NLP or cross-domain recommendation systems.

Relation to Neural Networks. For neural networks with m output units, the output function $f_\theta : \mathcal{X} \to \mathbb{R}^m$ is vector-valued. At initialization, an infinitely-wide network induces a Gaussian process with a matrix-valued covariance kernel—the Neural Network Gaussian Process (NNGP) kernel. During training, the dynamics are governed by the Neural Tangent Kernel (NTK), also a matrix-valued kernel. Understanding these as instances of vector-valued RKHS theory provides a unified framework connecting classical kernel methods with modern deep learning.

5 Conclusion

This report has surveyed the mathematical foundations of the Neural Tangent Kernel, developing the necessary background in Reproducing Kernel Hilbert Spaces and Gaussian Processes before deriving the NTK formulation itself. The central result is that infinitely wide neural networks trained by gradient descent behave as kernel machines, with training dynamics governed by a deterministic kernel that remains essentially constant throughout learning.

The NTK framework provides valuable theoretical insights. It explains why gradient descent can converge globally in overparameterized networks and offers exact characterizations of training dynamics in the infinite-width limit. The connections to classical kernel methods—through vector-valued RKHS and the representer theorem—reveal deep mathematical structure underlying neural network optimization.

However, NTK theory has clear limitations. It does not explain generalization, and it assumes frozen features rather than the hierarchical representation learning observed in practice. Empirical evidence shows that finite-width networks consistently outperform kernel methods, suggesting that practical deep learning derives its power from operating outside the lazy regime that NTK characterizes.

Understanding this gap remains an open challenge. NTK establishes a rigorous baseline for the infinite-width limit, but the mechanisms enabling feature learning at realistic widths require further investigation. This survey has aimed to present the mathematical tools needed to understand NTK theory and to clarify both what it explains and where its explanatory power ends.

Acknowledgments

The author acknowledges the use of AI tools for assistance with document preparation and formatting, as well as the core structure of the paper and connections between concepts are guided by the authors own understanding.

References

Mauricio A. Alvarez, Lorenzo Rosasco, and Neil D. Lawrence. Kernels for vector-valued functions: a review, 2012. URL https://arxiv.org/abs/1106.6251.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *CoRR*, abs/1806.07572, 2018. URL http://arxiv.org/abs/1806.07572.

Palle E. T. Jorgensen and James Tian. Operator-valued kernels, machine learning, and dynamical systems, 2024. URL https://arxiv.org/abs/2405.09315.

Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as gaussian processes. In *International Confer-*

ence on Learning Representations, 2018. URL https://openreview.net/forum?id=B1EA-M-0Z.

Radford M. Neal. *Bayesian Learning for Neural Networks*, volume 118 of *Lecture Notes in Statistics*. Springer-Verlag, New York, 1996. ISBN 0-387-94724-8.

Lilian Weng. Some math behind neural tangent kernel. *Lil'Log*, Sep 2022. URL https://lilianweng.github.io/posts/2022-09-08-ntk/.